

Computer Network Defense: Docker Lab

Table of contents

1	Introduction	1
2	Installing Docker	2
3	Pulling and Running Your First Container	3
4	Exploring Container Isolation	4
5	Container Lifecycle	5
6	Sharing Files Between Host and Container	5
7	Cleaning Up	9
8	Wrap-Up Discussion	9

1 Introduction

A **Docker container** is an isolated environment that packages an application along with everything it needs to run. For example, it has its own OS userland (files and programs), libraries, and configuration. Containers are portable, meaning the same container can run identically on any machine that has Docker installed. This makes them popular both for software development and for security purposes, since a problem inside a container is, by default, kept separate from the host machine.

Compared to a virtual machine, containers are lightweight since they share the host OS kernel rather than running a full separate OS. The trade-off is that their isolation is weaker than a virtual machine's: because every container shares the host's kernel, a vulnerability in the kernel or a poorly configured container can allow a process to escape to the host. Compared to

a chroot jail, containers provide stronger isolation and come with a rich ecosystem of pre-built environments.

In this lab you will install Docker, pull a pre-built container image, run and interact with containers, and explore how isolation works in practice.

 Note

All commands in this lab should be run on your **virtual machine**, not your host machine.

2 Installing Docker

1. Update your package list.

```
sudo apt update
```

2. Install Docker.

```
sudo apt install docker.io
```

3. Start and enable the Docker service so it runs on boot.

```
sudo systemctl start docker  
sudo systemctl enable docker
```

4. Verify Docker is installed and running.

```
sudo docker --version  
sudo systemctl status docker
```

You should see a version number and a status showing the service is **active (running)**. To exit, press **q**.

5. Add your user to the **docker** group so you can run Docker commands without **sudo**.

```
sudo usermod -aG docker $USER
```

 Warning

You will need to **log out and back in** for this group change to take effect. Do that now before continuing.

! Important

Membership in the **docker** group is effectively equivalent to having **root access** on the host. Anyone in the group can start a container that mounts the host's entire filesystem (e.g. `docker run -v /:/host ...`) and modify any file on it. That is acceptable on a lab VM, but on a real system only trusted administrators should be added to this group.

3 Pulling and Running Your First Container

Docker uses **images** as blueprints for containers. Images are downloaded from a registry (Docker Hub by default). A **container** is a running instance of an image.

1. Pull the official Ubuntu image from Docker Hub.

```
docker pull ubuntu
```

You should see Docker downloading the image layers. Once complete, the image is stored locally.

2. Verify the image was downloaded.

```
docker images
```

You should see **ubuntu** listed with its tag, size, and image ID.

3. Run the Ubuntu container interactively.

```
docker run -it ubuntu bash
```

- `-i` keeps standard input open (interactive)
- `-t` allocates a terminal
- `bash` is the command to run inside the container (optional in this case)

Your prompt will change to something like `root@<container_id>:/#`, indicating you are now inside the container.

! Group Discussion

You are now inside an isolated Ubuntu environment running on top of your Linux VM. Discuss with your group:

1. Run `whoami`. What user are you inside the container? Is this concerning? Why or why not?
2. Run `ls /`. How does the filesystem inside the container compare to what you'd

- expect on a normal Linux system?
3. Run `cat /etc/os-release`. What OS is the container running?

4 Exploring Container Isolation

While still inside the container, work through the following to understand how isolated it is from your host VM.

1. Try to install a package inside the container.

```
apt update
apt install tree
```

This works. You have a full package manager available inside the container.

2. Create a file inside the container.

```
echo "hello from inside the container" > /tmp/containerfile.txt
cat /tmp/containerfile.txt
```

3. Exit the container.

```
exit
```

4. Back on your VM, check whether the file you created exists on the host.

```
cat /tmp/containerfile.txt
```

It should not exist. The container's filesystem is isolated from the host.

! Group Discussion

1. You installed `tree` inside the container. Is it installed on your host VM now? Test it: run `which tree` on your host.
2. What does this tell you about the usefulness of containers for safely running untrusted software?
3. What would happen to all of your changes inside the container if you ran `docker run -it ubuntu bash` again? Would the file still be there?

5 Container Lifecycle

1. List all running containers.

```
docker ps
```

Since you exited the container, nothing should be running.

2. List all containers, including stopped ones.

```
docker ps -a
```

You should see your previous container listed with a status of **Exited**.

3. Run a container in the background (detached mode) using the **-d** flag. This container will just sleep for 60 seconds.

```
docker run -d ubuntu sleep 60
```

4. Check that it is running.

```
docker ps
```

You should see it listed with a status of **Up**.

5. Stop the container using its container ID (the first column from `docker ps`).

```
docker stop <container_id>
```

6 Sharing Files Between Host and Container

Containers are isolated, but you can deliberately share a folder between the host and a container using a **volume mount**.

1. Create a directory on your VM to share.

```
mkdir ~/shared  
echo "this file is from the host" > ~/shared/hostfile.txt
```

2. Run a container with the shared folder mounted. The **-v** flag maps a host path to a container path.

```
docker run -it -v ~/shared:/data ubuntu bash
```

3. Inside the container, navigate to the mounted folder and view its contents.

```
cd /data
ls
cat hostfile.txt
```

You should see the file you created on the host.

4. Create a new file from inside the container.

```
echo "this file is from the container" > /data/containerfile.txt
exit
```

5. Back on your host, check the shared folder.

```
ls ~/shared
cat ~/shared/containerfile.txt
```

The file created inside the container should now exist on the host.

! Group Activity

Work with your group to SSH directly into a running Docker container.

On the host VM (one group member):

1. Start a container with port 22 exposed and keep it running.

```
docker run -d --name sshbox -p 2222:22 ubuntu sleep infinity
```

Arguments

- `-d` is for **detached mode**. This allows the container to run in the background so your terminal stays free to use for other commands for now.
- `--name sshbox` is used to give the container a name so you can reference it by name instead of ID.
- `-p 2222:22` forwards port 2222 on the host to port 22 inside the container (22 for ssh).
- `sleep infinity` is used to keep the container alive with no workload.

2. Install and configure SSH inside the container by running each of the following.

- a. First access the container.

```
docker exec -it sshbox bash
```

- b. Now install openssh-server. Follow any prompts while installing.

```
apt update  
apt install -y openssh-server
```

- c. Create a new user and set a password for them. We will use `-m` to ensure the user gets a home directory and `-s` to explicitly set the default shell for the user as bash.

```
useradd -m -s /bin/bash student  
passwd student
```

You will be prompted to enter and confirm a password for the user named **student**. Remember that you won't be able to see what you are typing when you're typing a password to protect against shoulder surfing.

- d. Start the SSH service and exit the container.

```
service ssh start  
exit
```

3. Find your VM's IP address.

```
hostname -I
```

On another VM (second group member):

4. SSH into the container by using the user name in the container, the VM's IP address, and specify port 2222 which is being forwarded to the container's port 22.

```
ssh student@<vm-ip> -p 2222
```

5. Once inside, verify you are in the container, not the VM, although the bash prompt should already be displaying this information.

```
cat /etc/hostname  
hostname
```

Compare this to the host's hostname.

! Group Activity: Docker and Your Firewall

In the Firewall Lab, you saw that UFW blocks all incoming connections by default. Let's see what happens to the container's SSH port when the firewall is on.

On the host VM (first group member):

1. Enable UFW. Do **not** add any rules.

```
sudo ufw enable
sudo ufw status verbose
```

Confirm that incoming traffic is denied by default and that there is no rule allowing port 2222.

On another VM (second group member):

2. Try to SSH into the host VM itself on port 22. This should hang and eventually time out, since the firewall is dropping incoming connections.

```
ssh <host_username>@<vm-ip>
```

Press **Ctrl+C** if you do not want to wait for the time out.

3. Now try to SSH into the container on port 2222 again.

```
ssh student@<vm-ip> -p 2222
```

Did it work?

Why this happens: when you publish a port with **-p**, Docker adds its own rules directly to the kernel's packet filter. Traffic to a published port is redirected to the container before it ever reaches the UFW rules that protect the host. In other words, **ports published by Docker bypass UFW**. This is a common real-world misconfiguration: an administrator believes the firewall is protecting a service that is actually exposed to the whole network. To limit exposure, publish ports only on the interface you need, e.g. **-p 127.0.0.1:2222:22** makes the port reachable only from the host itself.

On the host VM:

4. Disable UFW again so it does not affect the rest of this lab.

```
sudo ufw disable
```

7 Cleaning Up

Stopped containers and downloaded images take up disk space. Clean up when you are done.

1. Stop the `sshbox` container, which is still running in the background. A running container cannot be removed.

```
docker stop sshbox
```

2. Remove all stopped containers.

```
docker container prune
```

You will be prompted to confirm. Type `y` and press Enter.

3. Verify no containers remain.

```
docker ps -a
```

If containers still remain, stop them using their name or id and then run `docker container prune` again.

```
docker stop <container_id>
```

4. Remove the Ubuntu image.

```
docker rmi ubuntu
```

5. Verify the image was removed.

```
docker images
```

6. Remove the shared folder created during this lab.

```
rm -rf ~/shared
```

8 Wrap-Up Discussion

! Group Assignment

Before you leave, discuss the **Docker Lab** questions in the *Computer Network Defense: Assignment* document with your group and write down your answers. Your group will submit the answers for all of the labs together, as a single PDF on Canvas, after the final lab.